Cameron Lewis

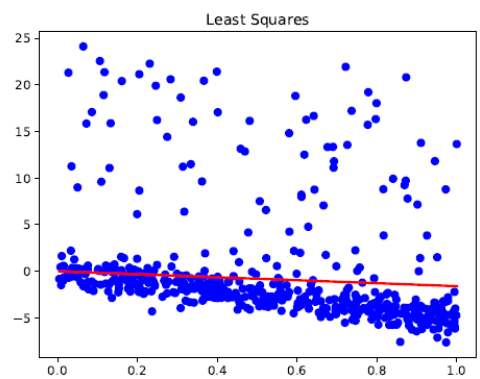
CAP4611

Oct 7th 2025

5593788

Assignment 3

1. Matrix Submission Format
   1.1. Converting to Matrix/vector/Norm Notation
      1.1.1. $\|Xw - y\|^2$
      1.1.2. $(Xw - y)^T V (Xw - y) + \frac{\lambda}{2} w^T w$
      1.1.3. $\|Xw - y\|_\infty + \frac{1}{2} 1^T A |w|$
   1.2. Minimizing Quadratic Functions as Linear Systems
      1.2.1. $w = v$
      1.2.2. $(X^T X + A)w = X^T y$
      1.2.3. $(X^T V X + \lambda I)w = X^T V y + \lambda w^0$
2. Robust Regression and Gradient Descent
   2.1. Weighted Least Squares in One Dimension




```
V = np.diag(v)

# Solve weighted normal equations
self.w = solve(X.T @ V @ X, X.T @ V @ y)
```

   2.2. Smooth Approximation to the L1-Norm
      2.2.1. $r_i = w^T x_i - y_i$

$$f_i(w) = \log(\exp(r_i)) + \exp(-r_i)$$

$$\frac{df_i}{dw} = \frac{1}{\exp(r_i) + \exp(-r_i)} \cdot (\exp(r_i) - \exp(-r_i) \cdot \frac{dr_i}{dw}$$

$$= \tanh(r_i)x_i$$

$$\nabla f(w) = \sum_{i=1}^{n} tanh(w^T x_i - y_i) \cdot x_i$$

2.3. Gradient Descent: Understanding the Code

2.3.1. alpha

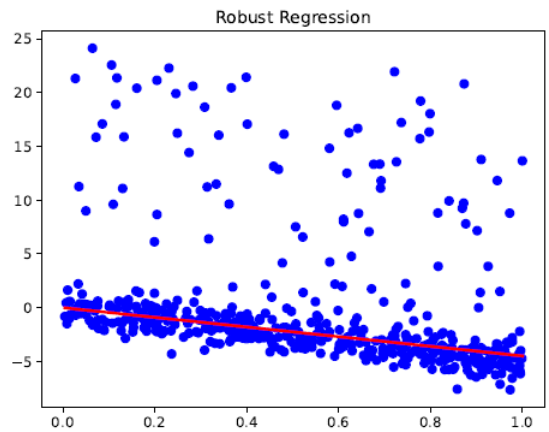
2.3.2. g_old

2.3.3. self.parameters

2.3.4. checks whether the optimization should terminate by testing if the gradient norm is below tolerance or if the max number of evaluations has been reached

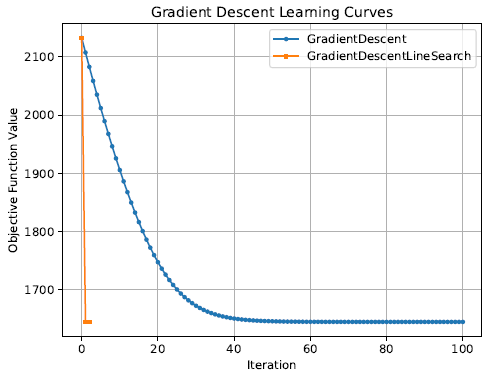
2.4. Robust Regression

2.4.1.




```
w = ensure_1d(w)
y = ensure_1d(y)

w = ensure_1d(w)
y = ensure_1d(y)

# Compute residuals
residuals = X @ w - y

# Compute function value using log-sum-exp
f = np.sum(np.log(np.exp(residuals) + np.exp(-residuals)))

# Compute gradient using tanh
g = X.T @ np.tanh(residuals)

return f, g
```

2.4.2.

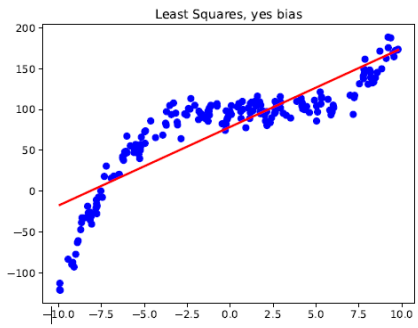


Line search seems to be efficient

3. Linear Regression and Nonlinear Bases

3.1. Adding a Bias Variable

3.1.1.



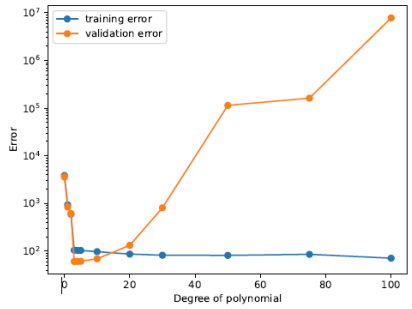

```
class LeastSquaresBias:
    "Least Squares with a bias added"

    def fit(self, X, y):
        n = X.shape[0]
        # Add column of ones to the left of X for bias term
        X_bias = np.hstack([np.ones((n, 1)), X])

        # Solve normal equations with augmented X
        self.w = solve(X_bias.T @ X_bias, X_bias.T @ y)

    def predict(self, X_pred):
        n = X_pred.shape[0]
        # Add column of ones to match training
        X_bias = np.hstack([np.ones((n, 1)), X_pred])

        return X_bias @ self.w
```

Training error = 938.4

Validation error = 844.4

3.2. Polynomial Basis

3.2.1.




```
def fit(self, X, y):
     Z = self._poly_basis(X)

     # Fit least squares on transformed featureas
     self.leastSquares.fit(Z, y)


def predict(self, X_pred):
    Z_pred = self._poly_basis(X_pred)

    # Predict using least squares model
    return self.leastSquares.predict(Z_pred)

# A private helper function to transform any X with d=1 into
# the polynomial basis defined by this class at
initialization.
# Returns the matrix Z that is the polynomial basis of X.
def _poly_basis(self, X):
    n = X.shape[0]
    # Initialize Z with appropriate size
    Z = np.zeros((n, self.p + 1))

    # Fill each column
    for j in range(self.p + 1):
        Z[:, j] = (X[:, 0] ** j)

    return Z
```

As p increases, the training error decreases, meaning that higher-degree polynomials can fit the training data more closely, and the training error becomes very small.

4. Very Short Answer Questions

4.1. K-means will assign it to the nearest cluster and density will label it as noise

4.2. K-means is initialization dependent, and random restarts can reduce bad starting points, but density-based is the same output every time so it doesn't matter

4.3. Yes. Depending on how points are linked it can form long irregular cluster shapes

4.4. Fit a Gaussian bell curve to data and the problem is the data wont follow the gaussian shape
4.5. KNN graph with outliers being points with few neighbors or far distances, and the problem is that the result will be heavily dependent on k
4.6. Train a classifier on normal and abnormal data. The problem is that it needs labeled examples of outlier types
4.7. It would not make sense since gradient descent is better used for large datasets
4.8. The column lets the model learn an intercept and decision trees do not need this
4.9. We still need gradient descent because of the loss function
4.10. Too small the algorithm learns to slowly and when its too large it jumps around too much
4.11. It allows us to compute gradients and use gradient descent effectively by making sharp non smooth functions smooth
4.12. Use sin and cos for different values of *k* to capture patterns in the data.